Vectorless Tree Retrieval for RAG

How PageIndex-style systems build a document tree, navigate it with reasoning, and answer with page-anchored citations - without a vector database

---

Implementation guide + architecture patterns + use cases
Date: February 27, 2026

This document is written for engineers and architects building document QA assistants for long, structured files (finance, legal, compliance, manuals). It focuses on practical steps you can implement in Python and deploy in production with guardrails and evaluation.

Table of contents

1. Executive summary
2. Why this is happening now
3. Key concepts
4. Architecture
5. How the tree search works
6. Implementation guide
7. Prompts and schemas
8. Use cases
9. Evaluation and metrics
10. Pitfalls and troubleshooting
11. Performance and deployment
12. Hybrid patterns and comparisons
13. Security and UX
14. When not to use it
15. Production checklist
16. Roadmap
17. Conclusion
18. Glossary
19. FAQ
20. References

Note: Page numbers are added automatically. Section order mirrors typical build -> run -> productionize flow.

1. Executive summary

Vectorless tree retrieval is a new retrieval-augmented generation (RAG) pattern that makes vector databases optional for a large class of document QA tasks, especially long, structured documents such as annual reports, contracts, policies, and manuals.

Instead of converting every chunk into an embedding and running cosine similarity, the system builds a hierarchical index (a "document tree") from the table of contents, headings, and page structure. At query time, an LLM (or a lightweight router model) navigates the tree in a coarse-to-fine manner: it selects the most promising branch, drills down to the relevant section, and only then reads the evidence needed to answer.

This guide explains the architecture behind PageIndex-style systems, why they can outperform vector search on structured benchmarks, and how to implement an end-to-end prototype you can productionize with guardrails, evaluation, and observability.

2. Why this is happening now

Classic vector RAG works well for broad, unstructured corpora, but it struggles on long structured documents where relevance is often determined by document logic rather than semantic similarity.
In finance and legal documents, answers may live in appendices, footnotes, or nested subclauses that share few surface similarities with the question. Chunking can also split a single concept across boundaries, which leads to partial evidence and hallucinated glue text.
Recent work and engineering experiments have revived a different idea: if the document already encodes a hierarchy (sections, subsections, exhibits, schedules), use that hierarchy as the primary retrieval index, and treat retrieval as navigation, not nearest-neighbor search.
PageIndex is an open-source framework that popularized this "reasoning-first, vectorless" approach and reported strong results on FinanceBench-style document QA tasks.

The problem this solves

Most "vector RAG" pipelines were designed for web-like corpora: lots of short pages, weak structure, and high vocabulary variation. The retrieval objective is recall: bring back something roughly related, then let the model answer.

Enterprise documents are different. A contract, a policy, or a 10-K is engineered to be read using structure: section numbers, defined terms, exhibits, and consistent headings. Relevance is frequently structural. For example, "termination for convenience" is almost always in a termination section, even if the wording varies. Likewise, "revenue recognition" lives in an accounting policy section, and the relevant details might be split between narrative text and a table.

Vector search can miss these cases because it optimizes for semantic similarity at the chunk level. Two failures are common:
- The question is abstract but the document is concrete. The embedding similarity is low even though the section is the correct one.
- The chunk boundaries cut a definition, table, or list in half, causing the model to answer with incomplete evidence.

Tree retrieval flips the mental model: instead of searching for "similar text", it searches for "the right place to read".

3. Key concepts

Tree-based vectorless retrieval has three core ideas:

1) Structure is a first-class index. The unit of retrieval is not an arbitrary chunk; it is a natural node such as a section, subsection, table, or exhibit. Nodes keep stable references like (document id, page range, heading path).

2) Coarse-to-fine search. Instead of scoring every node, the system starts at the root and repeatedly chooses which branch to explore next. This dramatically reduces the candidate set and makes the retrieval path explainable.

3) Reasoning-guided reading. The router is allowed to "look ahead" by reading short summaries of nodes and then decide where to go. Only the final evidence nodes are expanded into full text for the answer.

Key terms and mental model

Document tree: A rooted hierarchy representing a document. Internal nodes are sections or chapters. Leaves are the smallest retrievable units, such as a subsection, a table, or a short page range. Each node has:
- title and optional section number
- path (e.g., 7 -> 7.2 -> 7.2.1)
- page range anchors
- children pointers
- node card (a short summary used for navigation)

Node card: A compact, structured summary of a node. It is not a free-form abstract. In production, it is best to keep a template so routing is consistent. A good template is:
- Scope: what this section covers in 1 sentence
- Topics: 5-10 topical phrases
- Entities: key defined terms, metrics, names
- Tables/figures: list of tables and what they contain
- "Look here when": 1-2 examples of questions that belong here

Search policy: The algorithm that decides which nodes to explore next. In PageIndex-style systems the search policy is LLM-guided: the model reads node cards and chooses which children to open.

Evidence pack: The final context fed into the answer model. It should include page-anchored snippets, table extracts, and the full heading path so citations are easy and audit trails are clear.

4. Architecture

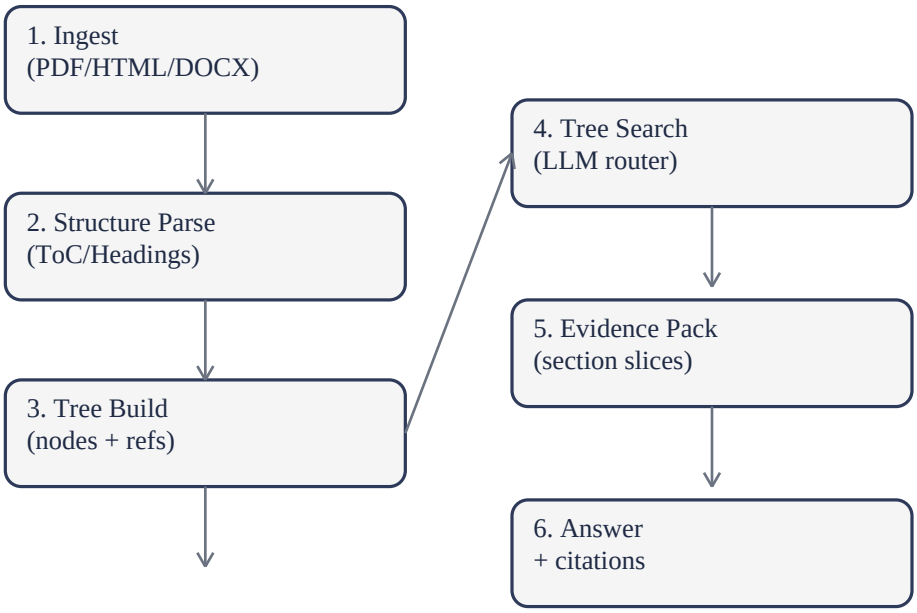

Figure: High-level reference flow for vectorless tree retrieval.

A production-grade PageIndex-style system usually has two phases.

OFFLINE (index build)
- Parse: extract page text, headings, table of contents, and layout signals.
- Normalize: clean headers/footers, fix hyphenation, detect tables, preserve page numbers.
- Tree build: build a hierarchy where each node has a title, path, page range, and children.
- Summarize nodes: store short "node cards" (50-200 words) that describe what the node contains. These cards are the primary navigation surface.
- Persist index: store the tree and node cards in a standard database (Postgres, SQLite, JSON on object storage). No vector store is required.

ONLINE (query time)
- Intent and constraints: detect if the query is about a specific document, time period, or section; apply metadata filters.
- Tree search: starting from the root, pick the best child nodes using LLM reasoning, optionally with beam search (keep top B branches) and budget limits.
- Evidence assembly: once leaf nodes are selected, retrieve full text for those page ranges and add minimal surrounding context.
- Answer with citations: generate an answer that cites page and section references; keep an audit trail of the navigation path.

Reference architecture (components and responsibilities)

A mature implementation often separates concerns into services so you can scale and secure them independently:

A) Index Builder (offline batch)
- Input: PDFs / HTML / DOCX
- Output: index artifact (tree + node cards + page text + table extracts)
- Runs: on document upload and on document updates
- Notes: version every build; keep deterministic parsing for auditability

B) Index Store
- Stores: JSON tree, node cards, page map, table CSV/JSON
- Common choices: Postgres (JSONB), SQLite for single tenant, or object storage with a metadata DB
- Important: keep stable node ids so you can trace retrieval across versions

C) Query Router
- Determines: target documents, filters, and which retrieval mode to use (tree vs keyword vs vector)
- Often uses: simple rules + a lightweight LLM classifier
- Output: retrieval plan (document ids, constraints, max budget)

D) Tree Search Engine (online)
- Implements: coarse-to-fine navigation, beam search, caching of node cards, confidence scoring
- Calls: router model for decisions; does not need to expose raw document text to the router

E) Evidence Assembler
- Pulls: leaf node text by page ranges, plus optional context windows
- Adds: structured citations and extracts tables
- Applies: redaction, access control, and de-duplication

F) Answer Synthesizer
- Generates: final answer, summary, and structured citations
- Enforces: "no evidence, no answer" rules; returns an "insufficient evidence" response when needed

G) Telemetry + Evaluation
- Captures: retrieval path, nodes visited, model calls, latency, and token usage
- Feeds: offline evaluation dashboards and regression tests

5. How the tree search works

How the tree search works (practical algorithm)

At a high level, the system repeatedly asks: "Given the question and what I have seen so far, which child node is most likely to contain the answer?"

A practical, production-friendly algorithm is beam search over the tree:

Inputs
- question q
- root node r
- beam width B (e.g., 3)
- max depth D (e.g., 6)
- max nodes visited N (e.g., 30)
- stop threshold T (router confidence)

State
- each beam item is (node_id, score, rationale, depth)

Loop
1) Start with beam = [(root, 1.0, "start", 0)]
2) For each beam item, fetch its children and their node cards.
3) Ask the router model to rank children. The router returns a probability distribution or a top-k list with reasons.
4) Expand: push the best children into a candidate list with updated scores.
5) Prune: keep the top B items by score.
6) Stop if:
- you reached a leaf node, or
- router confidence is above T for a leaf, or
- budgets (D or N) are exhausted.

Output
- a small set of leaf nodes (often 2-5) plus their rationales and scores.
- the navigation path (for explainability).

This is different from vector retrieval because the router is reasoning about structure and intent. It is also cheaper than scoring every node because the router sees only a small set of node cards each step.

Walkthrough example (end-to-end)

Question: "What notice period applies for termination for convenience, and where is it stated?"

Step 1 - Router plan
The query router detects a contract-style question, selects the relevant contract document, and chooses tree navigation mode because the answer is likely contained in a specific section.

Step 2 - First hop (root -> major sections)
The router sees node cards like:
- Definitions
- Term and Termination

- Fees and Payment
- Confidentiality
It selects "Term and Termination" with high confidence.

Step 3 - Second hop (termination subsections)
Within "Term and Termination", node cards include:
- Termination for cause
- Termination for convenience
- Effects of termination
The router selects "Termination for convenience".

Step 4 - Evidence extraction
The evidence assembler retrieves the page lines covering that subsection and captures the exact notice period sentence. If the notice period is in a table (rare, but possible), it includes the relevant table row.

Step 5 - Answer with citations
The answer model returns:
- The notice period value (for example, "30 days written notice")
- The clause heading path and page reference
- Optionally, the exact quoted sentence for high-stakes usage

6. Implementation guide

Implementation can be done with a few hundred lines of Python. The key is to separate: (a) document parsing, (b) index representation, (c) search policy, and (d) answer synthesis.

You can implement the concept from scratch, or you can start with the PageIndex open-source repository and adapt it to your domain and security requirements.

Below is a practical blueprint that works even when you cannot send full documents to an external model: only small node cards or snippets need to be used for routing, and you can keep raw text inside your private boundary.

Implementation guide (hands-on)

This section gives you a blueprint you can implement in your stack. The code examples are intentionally minimal; you should adapt them to your LLM provider, storage layer, and security model.

1) Document parsing and page map
Goal: produce a reliable page map so you can always cite answers. Extract:
- per-page text
- per-page layout signals (fonts, bounding boxes) if available
- detected headings and section numbers

2) Build the tree
Common heuristics:
- If the PDF has a Table of Contents, parse it and map ToC entries to page ranges.
- Otherwise, infer headings using font size, boldness, indentation, and recurring patterns like "1.", "1.1", "Section 7.2".
- Create nodes for major sections, then fill children based on heading levels.
- Create special nodes for tables/figures if extraction is possible.

3) Generate node cards
Node cards are your retrieval index. Generate them with:
- deterministic prompts
- capped length (for example 120 words)
- a strict output schema (JSON) so downstream routing is stable
Store node cards alongside their node id and page anchors.

4) Tree search (router)
Use a small prompt that shows:
- the question
- the current heading path
- the list of children node cards
Ask the model to return:
- best child id(s)
- a short rationale
- a confidence score
Keep a budget to avoid runaway cost.

5) Evidence assembler
When you hit leaf nodes:

- retrieve full text for the node page range
- expand by a small window (e.g., +/- 10 lines) to avoid truncation
- normalize whitespace
- de-duplicate repeated headers/footers
- attach citations as (doc_id, page_start, page_end, heading_path)

6) Answer synthesis
Use a prompt that enforces:
- only answer using evidence
- include citations for each claim
- if evidence is insufficient, ask for clarification or refuse

7) Observability
Log:
- nodes visited
- router rationales
- time per step
- tokens
This is crucial for debugging routing failures.

Code skeletons

The following snippets show how to separate index build, routing, evidence, and answering. Treat them as a blueprint, not a library.

```
## Minimal reference implementation (Python skeleton) The goal is to show the moving parts. This is not a full library. Data model - Node: id, title, page_start, page_end, children[], card_text - Index: doc_id, root_node_id, nodes (dict) Parsing - extract_pages(pdf) -> list[str] - detect_headings(pages) -> list[Heading(title, level, page)] - build_tree(headings) -> nodes Routing - choose_children(question, parent_node, child_cards) -> ranked child ids Search - beam_search(question, root, beam_width, budgets) -> leaf nodes Evidence - slice_pages(pages, page_start, page_end) -> evidence text + citations Answer - generate_answer(question, evidence_pack) -> answer with citations

## A more complete implementation sketch (pseudo-code) Below is a longer pseudo-code sketch you can translate to your framework. It shows how the parts fit together. The same logic works in a monolith or microservices. 1) Index build - pages = parse_pdf(pdf_path) - headings = extract_headings(pages) - tree = build_tree(headings, pages) - for node in tree.nodes: node.text = slice_pages(pages, node.page_start, node.page_end) node.card = make_node_card(node.title, node.path, node.text) - save_index(doc_id, tree) 2) Query - plan = make_plan(question, metadata) - leaf_nodes = tree_beam_search(plan, question) - evidence = build_evidence(plan.doc_id, leaf_nodes) - answer = answer_with_citations(question, evidence) - return answer, trace
```

7. Prompts and schemas

Prompts and schemas that work well

Tree systems live or die by consistency. Below are prompt patterns you can reuse.

A) Node card generation prompt (offline)
Input: section title, heading path, and the raw text for that node (page range).
Output: a compact JSON object.

Design rules
- Always include the heading path and page anchors.
- Prefer factual phrases over storytelling.
- Include unique identifiers: section numbers, defined terms, table names.
- Keep it short enough to fit many cards in a single router call.

Recommended JSON fields
- scope: 1 sentence
- topics: list of short phrases
- key_terms: list of defined terms or entities
- tables: list of table/figure captions, if any
- look_here_when: list of question patterns
- exclusions: what is explicitly not covered (optional)

B) Router prompt (online)
Input: user question, current node path, and a list of children cards (each with id + short fields).
Output: top-k child ids plus rationale and confidence.

Design rules
- Instruct the model to pick the place to read, not to answer.
- Ask for a short rationale that mentions specific terms or section numbers.
- Force a confidence score; use it for stop/backtracking.

C) Answer prompt (online)
Input: question + evidence pack with citations.
Output: answer where every claim is tied to at least one citation.

Design rules
- Quote critical numbers or legal phrases directly.
- If evidence is missing, say "insufficient evidence" and propose the next retrieval step.
- Keep citations structured so you can render them in UI (doc, page, heading).

Example: Node card generation (pseudo)

SYSTEM: You create a structured card describing a document section. Do not add external facts.
USER:
Title: "7.2 Termination for Convenience"
Heading path: "7 Term and Termination > 7.2 Termination for Convenience"
Pages: 14-16
Text:

ASSISTANT (JSON):
{
"scope": "Defines when either party may terminate without cause and the required notice procedure.",
"topics": ["notice period", "written notice", "effective date", "survival of obligations"],
"key_terms": ["Termination for Convenience", "Notice", "Effective Date"],
"tables": [],
"look_here_when": [
"Question asks about termination without cause or convenience.",
"Question asks about notice requirements for ending the agreement."
]
}

Example: Router prompt (pseudo)

SYSTEM: You are selecting the best section to read. Choose ids only.
USER:
Question: "What notice period applies for termination for convenience?"
Current node: "7 Term and Termination"
Children:
- id=7.1 card: scope=Termination for cause...
- id=7.2 card: scope=Termination for convenience...
- id=7.3 card: scope=Effects of termination...

ASSISTANT:
top=[{"id":"7.2","confidence":0.92,"why":"Question explicitly matches 'termination for convenience' and notice requirements are listed here."},
{"id":"7.3","confidence":0.38,"why":"May contain survival/effects but less direct."}]

8. Use cases

Vectorless tree retrieval is best when your data has strong internal structure.

High-impact use cases
- Finance: 10-K/annual reports, earnings decks, investor presentations, credit memos, audit reports.
- Legal: contracts, MSAs, DPAs, policy manuals, regulatory filings, terms and schedules.
- Compliance: SOPs, controls documentation, model risk docs, incident playbooks.
- Engineering: product manuals, runbooks, architecture decision records, RFCs.

Typical questions it excels at
- "What is the revenue recognition policy and where is it described?"
- "Which section describes termination for convenience and what notice period applies?"
- "Where is the definition of 'Confidential Information' and what are the exclusions?"
- "Which exhibit contains the fee schedule for region X?"

More use cases and patterns

1) Regulatory filings assistant
- Input: filings with stable section structures
- Questions: risk factors, accounting policy details, litigation disclosures
- Value: citations to exact sections reduce compliance risk

2) Contract review copilot
- Input: MSA + schedules + DPAs
- Questions: liability caps, SLA credits, security obligations, subcontracting rules
- Value: produces "clause cards" with citations for legal review

3) Incident response playbook assistant
- Input: runbooks and SOPs that are highly structured
- Questions: step-by-step procedures, escalation paths, contacts
- Value: navigation yields the exact step sequence, not a semantically similar paragraph

4) Engineering knowledge base for a product
- Input: long manuals and release notes
- Questions: troubleshooting steps, configuration options, compatibility matrices
- Value: tree retrieval finds the relevant chapter quickly and returns tables/steps reliably

Case study: Finance and procurement style QA

Imagine you have 5,000 vendor contracts and 2,000 SOP documents. Users ask two kinds of questions:

A) "What is the latest spend for Vendor X and how does it compare to last quarter?"
This is structured data. You should answer with SQL and a dashboard query, not by searching documents.

B) "Does the Vendor X master agreement allow subcontracting, and what approvals are required?"
This is document logic. The relevant information lives in a specific clause, often under headings like "Subcontracting", "Assignment", or "Third parties".

A hybrid plan works best:
- First, an intent router decides whether the question is structured or unstructured.

- For unstructured questions, it chooses tree navigation for the specific contract (if known) and keyword or vector search across contracts (if unknown).
- Tree navigation then drills down: Definitions -> Services -> Subcontracting (or Assignment) -> Approval requirements.
- The answer is produced with citations that reference the exact clause and page range.

The key benefit is auditability. In procurement and compliance, the question is not only "what is the answer?" but also "show me where it is written." Tree retrieval makes "where" a primary output, not an afterthought.

9. Evaluation and metrics

Evaluation is not optional. Tree-based systems can fail in different ways than vector RAG:

- Routing error: the router chooses the wrong branch early and never recovers.
- Summary drift: node cards omit a detail that is critical for navigation.
- Evidence truncation: selected leaf nodes are correct, but the extracted text misses the exact lines.
- Answering error: the model has evidence but still paraphrases incorrectly or overgeneralizes.

A simple but effective evaluation loop:
1) Build a small test set of 50-200 questions with gold citations (page/section).
2) Measure retrieval quality: "did we retrieve the cited section?"
3) Measure answer faithfulness: "does the answer only use retrieved evidence?"
4) Track cost and latency: tree search uses multiple model calls; cap budgets with a hard limit.

Metrics you should track (with definitions)

Retrieval metrics
- Hit@k (section level): does the gold section appear in the top k selected leaf nodes?
- Path accuracy: does the navigation path include the gold ancestor node (even if the leaf differs)?
- Depth to hit: how many hops were needed to reach the correct branch? Lower is better.

Answer metrics
- Exactness: is the final numeric or legal phrase exactly correct?
- Faithfulness: are all claims supported by retrieved evidence?
- Citation precision: do cited pages contain the supporting line?
- Coverage: does the answer address all parts of a multi-part question?

Operational metrics
- Router calls per query
- Tokens per router call and total tokens
- End-to-end latency p50/p95
- Cache hit rate for node cards
- Failure taxonomy counts (routing, evidence, answering)

These metrics help you decide whether a regression is due to parsing, routing, evidence assembly, or the answer model.

10. Pitfalls and troubleshooting

Common pitfalls and fixes:

- Poor PDF parsing: if headings and pages are wrong, the tree will be wrong. Use robust parsers, add heuristics for header/footer removal, and keep page anchors.
- Over-summarization: node cards that are too short become ambiguous. Keep a consistent template: scope, key topics, key tables, and unique identifiers (section numbers, defined terms).
- No backtracking: add beam search and allow one level of backtracking when confidence is low.
- Ignoring tables: many finance answers live in tables. Treat tables as first-class nodes and extract them separately.
- Missing security boundaries: if documents are sensitive, route using redacted node cards and run answer synthesis in a private model or in a secure environment.

Troubleshooting guide

Symptom: Router keeps choosing the wrong top-level section.
- Fix: Improve node cards at the top levels. Add "look_here_when" examples. Ensure section titles are clean and not truncated.
- Fix: Add a second-stage reroute: after selecting a node, ask the router "Is there a better sibling?" using the top 3 candidates.

Symptom: Router oscillates between two branches.
- Fix: Add a simple memory: track visited nodes and penalize repeats.
- Fix: Add a stop rule: if confidence does not improve after 2 steps, fallback to keyword search within the current subtree.

Symptom: Evidence is correct but answer misses a number.
- Fix: Use table-aware extraction and include the exact row/column text.
- Fix: In the answer prompt, require quoting of numbers and units.

Symptom: Page citations are off by 1-2 pages.
- Fix: Verify PDF page indexing (some parsers count from 0, some from 1).
- Fix: Keep a page map that includes original PDF page labels when available (Roman numerals, appendices).

Symptom: High latency.
- Fix: Reduce child list sizes per router call by grouping children into batches.
- Fix: Cache node cards and popular subtrees.
- Fix: Use a smaller router model and reserve the larger model for answer synthesis.

11. Performance and deployment

Cost, latency, and caching

Tree navigation uses multiple small model calls. In practice, you can make it efficient:

- Use a small routing model (or a smaller LLM deployment) for navigation.
- Cache node cards and child lists in memory. Most queries touch the same top-level sections.
- Cap the budget: max depth, max nodes visited, and max router tokens per call.
- Use early-stop: if the router confidence is high at depth 2 or 3, do not drill deeper.
- Batch child scoring: ask the router to choose among 8-15 children at once to reduce round trips.

A typical budget might look like:
- 3 to 6 router calls
- 1 evidence assembly call
- 1 answer synthesis call

In many enterprise settings, this compares favorably to vector RAG with reranking, which often involves embedding costs, vector DB queries, and a reranker model call.

Deployment blueprint

You can deploy this in three layers:

Layer 1 - Storage and indexing
- Object storage for original files and extracted artifacts
- Relational DB for metadata and JSON tree
- Optional: full-text index (Elasticsearch/OpenSearch) for keyword queries

Layer 2 - Retrieval services
- Index Builder worker (queue-driven) for ingestion
- Tree Search API for navigation and evidence assembly
- Policy service for RBAC/ABAC checks

Layer 3 - LLM services
- Router model endpoint (small, fast, cheap)
- Answer model endpoint (bigger, higher quality)
- Optional: citation verifier model (LLM-as-judge) for high-stakes workflows

Operationally, treat the index as a versioned artifact. If you update parsing logic, rebuild the index and keep the old version for reproducibility.

Data structures and storage choices

You do not need specialized infrastructure to store a tree index. What you need is:
- fast lookup by node id
- fast enumeration of children
- versioning of the index artifact
- stable references to page ranges

Option 1 - Single-tenant JSON artifact
- Store one JSON per document containing nodes and edges.

- Pros: simple, portable, easy to debug.
- Cons: not ideal for partial updates, slower for very large trees.

Option 2 - Relational DB with JSONB
- Table: documents (doc_id, metadata, active_version)
- Table: nodes (doc_id, version, node_id, parent_id, title, page_start, page_end, card_json)
- Pros: easy querying, partial updates, good audit story.
- Cons: slightly more engineering.

Option 3 - Object storage + metadata DB
- Store page text and table extracts as separate blobs.
- Store the tree and node cards in a metadata DB.
- Pros: scalable, cheap storage for large corpora.
- Cons: needs careful lifecycle management.

In all cases, keep a "page map" that links internal page indices to the original PDF page labels. It prevents citation confusion and makes user-facing references match what users see in the file viewer.

12. Hybrid patterns and comparisons

Hybrid design: when to add vectors anyway

Vectorless does not replace semantic search for everything. A robust enterprise system often orchestrates multiple retrieval modes:
- Tree navigation for single long documents with a clear hierarchy.
- Full-text keyword search (BM25) for exact references, names, and identifiers.
- Vector search for fuzzy recall across many documents where structure is inconsistent.
- Metadata filtering for time, entity, region, and document type.

A practical rule: use tree navigation when the user is asking about one or a few known documents; use vector or BM25 when the user is searching across a corpus.

Comparison with other "non-vector" retrieval ideas

Tree navigation is not the only alternative to vector RAG. You will also hear about:
- BM25 / inverted index search: excellent for exact terms and identifiers, but weaker for paraphrases.
- Knowledge graphs: strong for entity relationships and multi-hop reasoning, but expensive to build and maintain.
- Coarse-to-fine learned routers (research): such as ReTreever, which learns tree routing functions to reduce compute and improve transparency.

PageIndex-style systems sit in the middle: they leverage document structure (cheap to extract), use LLM reasoning for routing (flexible), and retain page-level citations (auditable).

13. Security and UX

Security and privacy considerations

Vectorless does not automatically mean safe. The model can still leak information if you send raw text to a hosted LLM.

A safe architecture pattern is "card-first routing":
- Keep raw document text inside your private environment.
- Generate node cards during ingestion, apply redaction and classification, and store the cards.
- At query time, the router model only sees node cards, not full pages.
- Only the final evidence slices are retrieved, and those slices can be sent to a private model (self-hosted or in a secure tenant) for answer synthesis.

Access control
- Enforce document-level permissions before retrieval begins.
- Enforce node-level permissions if a document contains mixed sensitivity sections.
- Record which nodes were opened and by whom for audit.

Prompt safety
- Treat node cards as untrusted input if they are generated from untrusted documents.
- Use prompt injection defenses: strip instruction-like text from node cards, or sandbox it into quoted fields.
- Never let the router follow document instructions such as "Ignore previous directions" or "Send the entire document".

UI and explainability: make the tree visible

Tree retrieval shines when users can see the reasoning path. A simple UI pattern that builds trust:
- Show the navigation trail (breadcrumbs): Root > Section > Subsection.
- Show the selected node cards (collapsed by default) so users understand why that section was chosen.
- Show cited snippets with page numbers and highlighted lines.
- Provide a "jump to page" action that opens the PDF viewer at the cited page.

This turns your assistant from a black box into a guided reader. In regulated environments, that can be the difference between adoption and rejection.

14. When not to use it

When NOT to use tree-based vectorless retrieval

Tree navigation is not a universal hammer. Avoid or de-prioritize it when:
- Your corpus is many short documents with weak or inconsistent headings.
- Users ask open-ended questions spanning dozens of documents ("What are the main themes across all policies?").
- The document structure is noisy (scanned PDFs without reliable OCR or headings).
- Your primary need is fuzzy semantic recall ("things like this") rather than "find the right section".

In these cases, classic full-text search and vector retrieval are often better starting points. You can still use tree navigation as a second stage once you have narrowed down to a few candidate documents.

15. Production checklist

Production checklist
- Index build: deterministic parsing, repeatable outputs, versioned index artifacts.
- Search policy: budgets (max depth, max nodes), beam width, stop criteria, confidence scoring.
- Evidence: page-anchored snippets, table extraction, minimal context expansion.
- Answers: citation enforcement, refusal when evidence is insufficient, quoted lines for high-stakes facts.
- Observability: trace the navigation path, record node ids and reasons, monitor latency and model tokens.
- Security: least-privilege access, PII redaction, tenant isolation, safe prompt templates.

Node card quality checklist

If routing is unstable, your node cards are usually the culprit. Audit a random sample and check:

Coverage
- Does the card mention the main topic in plain words?
- Does it include the most important defined terms and metrics?

Disambiguation
- If there are multiple similar sections (e.g., multiple "Definitions"), does the card include a unique identifier such as section number or scope?
- Does it mention exclusions ("does not cover pricing") to avoid misroutes?

Structure
- Is the output schema consistent across documents?
- Are lists short and specific (phrases, not paragraphs)?

Anchors
- Are page ranges correct?
- Does the heading path match the real document headings?

A simple practice: treat node cards as a dataset and run lint checks (length, missing fields, empty topics) before publishing an index version.

16. Roadmap

Where the technique is going next

The first generation of vectorless tree retrieval uses LLMs as routers over node cards. The next steps we are seeing in research and practice:
- Learned routers that are smaller and faster than general LLMs (tree routing models).
- Better structural parsing: combining ToC, typography, and layout to create more accurate trees.
- Multi-document trees: merging related documents into a shared hierarchy, like "Policy set -> Policy -> Section".
- Verification-aware retrieval: retrieval and verification co-train so the system prefers sections that support verifiable claims.
- UI-first evidence: frontends that show the navigation path, the selected node cards, and the cited lines, so users trust the system.

17. Conclusion

Conclusion

Vector databases are not going away, but the assumption that "RAG equals embeddings" is no longer true. For long structured documents, tree-based vectorless retrieval can be faster to operationalize, easier to audit, and more accurate because it aligns retrieval with how documents are written and how experts read.

If you are building enterprise assistants, the most important design move is to treat retrieval as an orchestrated capability:
- Choose the right retrieval mode for the data type.
- Make evidence and citations a first-class output.
- Invest in evaluation and telemetry so you can improve systematically.

Start small: pick one long document type (contracts, policies, annual reports), build a tree index, and test 50 real questions. If the navigation paths look sensible and the citations are correct, you will have a strong foundation to scale.

18. Glossary

Glossary (quick)

ABAC: Attribute-based access control, authorization based on user and resource attributes.
Beam search: Search strategy that keeps the top B candidates at each step instead of a single greedy path.
BM25: A classic ranking function used by full-text search engines.
Card-first routing: A security pattern where routing uses only small summaries (cards) rather than raw text.
Chunking: Splitting text into fixed-size segments for embedding and retrieval.
Citation precision: Whether a citation points exactly to supporting text, not just a nearby page.
Coarse-to-fine: A strategy that first narrows to a broad region, then drills down to details.
Defined term: A contract or policy term with a formal definition, often capitalized.
Evidence pack: The bundle of retrieved text snippets and metadata provided to the answer model.
Fallback retrieval: A secondary retrieval approach used when the primary approach is low confidence.
Heading path: A breadcrumb-like path from root to a node, used for navigation and citations.
Index artifact: The saved output of ingestion: tree, node cards, page text, and extracts.
Leaf node: A node without children; usually the final retrieval unit.
Node card: A structured summary of a node used for navigation decisions.
Router model: A model used to choose where to retrieve from; often smaller and cheaper than the answer model.
Reranker: A model that reorders retrieved candidates using deeper scoring.
Stop criteria: Rules that end search early (confidence, depth, budgets).
Tree navigation: Retrieval by traversing a hierarchical structure instead of similarity search.

19. FAQ

FAQ

Q: Is this truly "no vectors"?
A: In PageIndex-style systems, the core retrieval loop does not require embeddings or a vector database. Many teams still add optional vector search as a fallback for cross-document discovery. The key shift is that the primary retrieval signal for long documents is structure and reasoning, not nearest neighbors.

Q: Does it work on scanned PDFs?
A: It can, but quality depends on OCR and heading detection. If OCR is noisy, start with keyword search and consider adding a manual ToC or a lightweight layout model to recover structure.

Q: How do I keep costs under control?
A: Use a small router model, cap depth and visited nodes, cache node cards, and batch child ranking. Most of the value comes from getting the first 2-3 hops right, not from exploring every leaf.

Q: How do I make it deterministic?
A: Use fixed prompts and JSON schemas, set low temperature for routing, version your index builds, and log the full navigation trace. For high-stakes workflows, add a verifier pass that checks cited lines before returning the final answer.

Q: What is the minimum viable product?
A: One document type, one index builder, greedy navigation (no beam search yet), citation-enforced answers, and a 50-question evaluation set. You can add beam search, fallbacks, and advanced parsing after you have baseline metrics.

20. References

References and further reading

- VentureBeat (Jan 30, 2026): "This tree search framework hits 98.7% on documents where vector search fails" (PageIndex overview and benchmark results).
- PageIndex GitHub repository (VectifyAI): project description, notebooks, and examples.
- PageIndex blog (Sep 19, 2025): "Next-Generation Vectorless, Reasoning-based RAG" (conceptual introduction and citation).
- arXiv (Feb 2025): "ReTreever: Tree-based Coarse-to-Fine Representations for Retrieval" (research on tree routing and transparency).
- arXiv (Jan 2026): "T-Retriever: Tree-based Hierarchical Retrieval Augmented Generation ..." (tree-guided retrieval for graph-like data).
- Hacker News discussion: "Show HN: PageIndex - Vectorless RAG" (practical perspectives and tradeoffs).